

Table of Contents

Chapter 3 — Introduction to Python Programming	2
Complete Question Bank: MCQs & Short Questions	2
PART A — MULTIPLE CHOICE QUESTIONS (MCQs)	3
Section 1: Introduction to Python	3
Section 2: Python Syntax and Structure	5
Section 3: Variables and Data Types	7
Section 4: Arithmetic Operators	10
Section 5: Comparison Operators	12
Section 6: Assignment Operators	14
Section 7: Logical Operators	15
Section 8: Operator Precedence	16
PART B — SHORT ANSWER QUESTIONS	19
Section 1: Python & Programming Basics	19
Section 2: Syntax, Structure, and Comments	20
This program greets the user	21
Section 3: Variables and Data Types	22
Section 4: Arithmetic Operators	25
Section 5: Comparison Operators	26
Output: Did the student pass? True	27
Section 6: Assignment Operators	27
Section 7: Logical Operators	28
Section 8: Operator Precedence	30
Section 9: Mixed / Applied Questions	32
QUICK REFERENCE ANSWER KEY	35
MCQ Answer Key	35

Chapter 3 — Introduction to Python Programming

Complete Question Bank: MCQs & Short Questions

Total Questions: 80 MCQs · 60 Short Questions

Class: 10th Grade Computer Science

Topics Covered: Python Basics · Syntax & Structure · Variables · Data Types · Input/Output
· Arithmetic · Comparison · Assignment · Logical Operators · Operator Precedence

PART A — MULTIPLE CHOICE QUESTIONS (MCQs)

Circle or tick the correct option.

Section 1: Introduction to Python

Q1. Python was created by:

- (a) Bill Gates
 - (b) Dennis Ritchie
 - **(c) Guido van Rossum ✓**
 - (d) James Gosling
-

Q2. Python was named after:

- (a) A type of snake
 - (b) A Greek philosopher
 - **(c) Monty Python's Flying Circus (a TV show) ✓**
 - (d) A scientist named Python
-

Q3. Python is classified as a:

- **(a) High-level programming language ✓**
 - (b) Low-level programming language
 - (c) Machine language
 - (d) Assembly language
-

Q4. Which of the following best describes a "high-level" programming language?

- (a) It runs faster than machine code
 - **(b) It uses human-readable words instead of 0s and 1s ✓**
 - (c) It can only run on high-end computers
 - (d) It is difficult to learn
-

Q5. Python is used in which of the following fields?

- (a) Web development only
 - (b) Data analysis only
 - (c) Artificial Intelligence only
 - **(d) All of the above ✓**
-

Q6. The program that reads Python code and executes it line by line is called:

- (a) Compiler
 - **(b) Interpreter ✓**
 - (c) Assembler
 - (d) Linker
-

Q7. The default port used by Python's development server is NOT relevant here, but Python's standard file extension is:

- (a) `.pyt`
 - (b) `.pt`
 - **(c) `.py` ✓**
 - (d) `.python`
-

Q8. Which of the following is a free, cloud-based platform for writing Python code without any installation?

- (a) PyCharm
 - (b) VS Code
 - **(c) Google Colab ✓**
 - (d) IDLE
-

Q9. When installing Python on Windows, which important option must be checked?

- (a) Install for all users
 - (b) Create a desktop shortcut
 - **(c) Add Python to PATH ✓**
 - (d) Install pip separately
-

Q10. Which of the following is NOT a Python IDE?

- (a) PyCharm
 - (b) VS Code
 - (c) IDLE
 - **(d) MySQL Workbench ✓**
-

Q11. An IDE stands for:

- (a) Internal Development Engine
 - **(b) Integrated Development Environment ✓**
 - (c) Internet Data Exchange
 - (d) Instruction Design Engine
-

Q12. What does a computer program do?

- (a) Stores data permanently
 - (b) Connects to the internet
 - **(c) Gives precise, step-by-step instructions to a computer ✓**
 - (d) Designs hardware components
-

Section 2: Python Syntax and Structure

Q13. Which function is used to display output on the screen in Python?

- **(a) print() ✓**
 - (b) display()
 - (c) show()
 - (d) output()
-

Q14. What is the correct syntax to print "Hello World" in Python?

- (a) `echo("Hello world")`
 - **(b) `print("Hello World")` ✓**
 - (c) `Print("Hello World")`
 - (d) `printf("Hello World")`
-

Q15. What is the output of the following code?

```
print("Pakistan")
print("Zindabad")
```

- (a) Pakistan Zindabad
 - **(b) Pakistan (newline) Zindabad ✓**
 - (c) PakistanZindabad
 - (d) Error
-

Q16. Which symbol is used to write a single-line comment in Python?

- (a) `//`
 - (b) `/*`
 - (c) `--`
 - **(d) `#` ✓**
-

Q17. What happens when Python encounters a line starting with `#`?

- (a) It displays the line on screen
 - (b) It stores it in a variable
 - **(c) It ignores the line completely ✓**
 - (d) It causes an error
-

Q18. Which of the following is a valid single-line comment in Python?

- (a) `// This is a comment`
 - **(b) `# This is a comment` ✓**
 - (c) `<!-- This is a comment -->`
 - (d) `** This is a comment **`
-

Q19. What is the output of the following code?

```
# print("Hello")
print("World")
```

- (a) Hello World
 - (b) Hello
 - **(c) World ✓**
 - (d) No output
-

Q20. In Python, indentation refers to:

- (a) The font size of code
 - (b) The colour of code in the editor
 - **(c) The spaces at the beginning of a line that define code structure ✓**
 - (d) The number of lines in a program
-

Q21. What will `print("Hello", "World")` output?

- (a) HelloWorld
 - (b) Hello,World
 - **(c) Hello World ✓**
 - (d) "Hello" "World"
-

Section 3: Variables and Data Types

Q22. A variable in Python is best described as:

- (a) A fixed value that cannot change
 - **(b) A named container that stores a value in memory ✓**
 - (c) A mathematical formula
 - (d) A type of loop
-

Q23. What symbol is used to assign a value to a variable in Python?

- (a) `==`
 - (b) `:`
 - **(c) `=` ✓**
 - (d) `->`
-

Q24. What is the output of the following code?

```
age = 25
print("Age:", age)
```

- **(a) Age: 25 ✓**
 - (b) age: 25
 - (c) Age: age
 - (d) 25
-

Q25. Which of the following is a valid Python variable name?

- (a) `2score`
 - (b) `_score` ✓
 - (c) `score-1`
 - (d) `for`
-

Q26. Which of the following variable names is INVALID in Python?

- (a) `my_name`
 - (b) `name1`
 - (c) `1name` ✓
 - (d) `Name`
-

Q27. In Python, variable names are:

- (a) Case-insensitive
 - (b) **Case-sensitive** ✓
 - (c) Always lowercase
 - (d) Always uppercase
-

Q28. Which of the following cannot be used as a variable name?

- (a) `student`
 - (b) `_value`
 - (c) `result2`
 - (d) `while` ✓
-

Q29. What data type is the value `17` in Python?

- (a) **int** ✓
 - (b) float
 - (c) str
 - (d) bool
-

Q30. What data type is the value `3.14` in Python?

- (a) int
- (b) **float** ✓
- (c) str
- (d) bool

Q31. What data type is the value `"Lahore"` in Python?

- (a) int
 - (b) float
 - **(c) str ✓**
 - (d) bool
-

Q32. What data type is the value `True` in Python?

- (a) int
 - (b) str
 - (c) float
 - **(d) bool ✓**
-

Q33. Which of the following correctly stores a student's name?

- (a) `name = Ali`
 - **(b) `name = "Ali"` ✓**
 - (c) `name == "Ali"`
 - (d) `"name" = Ali`
-

Q34. What will the following code output?

```
x = 10
x = 20
print(x)
```

- (a) 10
 - **(b) 20 ✓**
 - (c) 10 20
 - (d) Error
-

Q35. Which function is used to take input from the user in Python?

- (a) `get()`
 - (b) `read()`
 - **(c) `input()` ✓**
 - (d) `scan()`
-

Q36. The `input()` function in Python always returns data as:

- (a) int
 - (b) float
 - **(c) str (string) ✓**
 - (d) bool
-

Q37. To store a user's age entered via `input()` as an integer, you should write:

- (a) `age = input("Enter age: ")`
 - (b) `age = float(input("Enter age: "))`
 - **(c) `age = int(input("Enter age: "))` ✓**
 - (d) `age = str(input("Enter age: "))`
-

Q38. What does `+` do when used between two strings in Python?

- (a) Adds their lengths
 - **(b) Joins (concatenates) the two strings ✓**
 - (c) Compares the strings
 - (d) Causes an error
-

Section 4: Arithmetic Operators

Q39. What is the output of `print(10 + 3)`?

- **(a) 13 ✓**
 - (b) 103
 - (c) 7
 - (d) 30
-

Q40. What is the output of `print(10 / 3)`?

- (a) 3
 - (b) 1
 - **(c) 3.3333333333333335 ✓**
 - (d) 4
-

Q41. What does the `///` operator do in Python?

- (a) Calculates the remainder
 - (b) Performs regular division
 - **(c) Performs floor division (discards decimal part) ✓**
 - (d) Raises a number to a power
-

Q42. What is the output of `print(10 // 3)`?

- **(a) 3 ✓**
 - (b) 3.33
 - (c) 1
 - (d) 4
-

Q43. What does the `%` operator return?

- (a) Percentage value
 - (b) Result of division
 - **(c) Remainder after division ✓**
 - (d) Floor of the number
-

Q44. What is the output of `print(10 % 3)`?

- **(a) 1 ✓**
 - (b) 3
 - (c) 0
 - (d) 3.33
-

Q45. What does the `**` operator do in Python?

- (a) Multiplies two numbers
 - **(b) Raises the left number to the power of the right ✓**
 - (c) Divides two numbers
 - (d) Returns the square root
-

Q46. What is the output of `print(2 ** 10)`?

- (a) 20
- (b) 100
- **(c) 1024 ✓**
- (d) 512

Q47. Which operator always returns a float in Python, even for whole-number division?

- (a) `//`
 - (b) `%`
 - (c) `/` ✓
 - (d) `**`
-

Q48. What is the result of `17 % 5`?

- (a) 2 ✓
 - (b) 3
 - (c) 5
 - (d) 12
-

Section 5: Comparison Operators

Q49. Comparison operators in Python always return:

- (a) An integer
 - (b) A string
 - (c) True or False (Boolean) ✓
 - (d) A float
-

Q50. Which operator checks if two values are equal in Python?

- (a) `=`
 - (b) `==` ✓
 - (c) `!=`
 - (d) `>=`
-

Q51. What is the output of `print(10 == 10)`?

- (a) True ✓
 - (b) False
 - (c) 10
 - (d) Error
-

Q52. What is the output of `print(10 != 5)`?

- **(a) True ✓**
 - (b) False
 - (c) 10
 - (d) 5
-

Q53. What is the output of `print(5 > 10)`?

- (a) True
 - **(b) False ✓**
 - (c) 5
 - (d) 10
-

Q54. What is the output of `print(10 >= 10)`?

- **(a) True ✓**
 - (b) False
 - (c) 0
 - (d) Error
-

Q55. What is the difference between `=` and `==` in Python?

- (a) They are the same
 - (b) `==` assigns a value; `=` compares
 - **(c) `=` assigns a value; `==` compares for equality ✓**
 - (d) `=` is used for strings; `==` for numbers
-

Q56. What is the output of `print(7 <= 7)`?

- **(a) True ✓**
 - (b) False
 - (c) 7
 - (d) 0
-

Section 6: Assignment Operators

Q57. What does `a += 5` mean?

- (a) `a = 5`
 - (b) `a = a - 5`
 - **(c) `a = a + 5` ✓**
 - (d) `a + 5`
-

Q58. If `a = 10`, what is the value of `a` after `a -= 3`?

- (a) 13
 - **(b) 7 ✓**
 - (c) 3
 - (d) 30
-

Q59. If `a = 4`, what is the value of `a` after `a *= 3`?

- (a) 7
 - (b) 43
 - **(c) 12 ✓**
 - (d) 1
-

Q60. If `a = 20`, what is the value of `a` after `a /= 4`?

- **(a) 5.0 ✓**
 - (b) 5
 - (c) 16
 - (d) 24
-

Q61. What does `a //= 3` do?

- (a) Multiplies a by 3
 - (b) Divides a by 3 and keeps the decimal
 - **(c) Divides a by 3 using floor division and stores the result ✓**
 - (d) Raises a to the power 3
-

Q62. If `a = 11`, what is the value of `a` after `a %= 4`?

- (a) 3 ✓
 - (b) 2
 - (c) 4
 - (d) 7
-

Section 7: Logical Operators

Q63. Which logical operator returns True only when BOTH conditions are True?

- (a) and ✓
 - (b) or
 - (c) not
 - (d) xor
-

Q64. Which logical operator returns True when AT LEAST ONE condition is True?

- (a) and
 - (b) or ✓
 - (c) not
 - (d) if
-

Q65. What is the output of `print(True and False)`?

- (a) False ✓
 - (b) True
 - (c) 0
 - (d) Error
-

Q66. What is the output of `print(True or False)`?

- (a) True ✓
 - (b) False
 - (c) 1
 - (d) Error
-

Q67. What is the output of `print(not True)`?

- (a) True
 - **(b) False ✓**
 - (c) 0
 - (d) 1
-

Q68. What is the output of `print(not False)`?

- **(a) True ✓**
 - (b) False
 - (c) not False
 - (d) Error
-

Q69. What is the output of `print(False and False)`?

- **(a) False ✓**
 - (b) True
 - (c) 0
 - (d) None
-

Q70. What is the output of `print(False or True)`?

- **(a) True ✓**
 - (b) False
 - (c) 1
 - (d) Error
-

Section 8: Operator Precedence

Q71. What is the output of `print(2 + 3 * 4)`?

- (a) 20
 - **(b) 14 ✓**
 - (c) 24
 - (d) 9
-

Q72. What is the output of `print((2 + 3) * 4)`?

- (a) 20 ✓
 - (b) 14
 - (c) 24
 - (d) 9
-

Q73. In Python operator precedence, which operation is performed FIRST?

- (a) Parentheses `()` ✓
 - (b) Addition `+`
 - (c) Multiplication `*`
 - (d) Exponentiation `**`
-

Q74. What is the output of `print(2 ** 3 + 1)`?

- (a) 2
 - (b) 16
 - (c) 9 ✓
 - (d) 7
-

Q75. What is the output of `print(15 / 3 * 2)`?

- (a) 10.0 ✓
 - (b) 2.5
 - (c) 5.0
 - (d) 30
-

Q76. What is the output of `print(15 // 4 + 2)`?

- (a) 5 ✓
 - (b) 5.75
 - (c) 6
 - (d) 3
-

Q77. Which has higher precedence in Python?

- (a) Addition
- (b) Subtraction
- (c) Multiplication ✓
- (d) Addition and Subtraction have equal precedence to Multiplication

Q78. What is the output of `print(4 + 10 / 2 - 11 % 3)`?

- (a) 6
 - (b) 8
 - **(c) 7.0 ✓**
 - (d) 5.0
-

Q79. Exponentiation (`**`) in Python is:

- (a) Left-associative
 - **(b) Right-associative ✓**
 - (c) Non-associative
 - (d) Same as multiplication
-

Q80. What is the output of `print(3 + 2 ** 2 * 5)`?

- (a) 100
- **(b) 23 ✓**
- (c) 27
- (d) 125

Explanation: `**` first → `2**2 = 4`, then `*` → `4*5 = 20`, then `+` → `3+20 = 23`

PART B — SHORT ANSWER QUESTIONS

Answer each question in 2–5 sentences. Where asked, include a code example.

Section 1: Python & Programming Basics

Q1. What is Python? Mention two fields where it is used.

Answer: Python is a high-level, interpreted programming language known for its simple and readable syntax. It is widely used in web development (to build websites and web applications) and in artificial intelligence and data analysis (to process large datasets and build machine learning models).

Q2. What is a programming language? Why do we need one?

Answer: A programming language is a set of rules and symbols that allows humans to write instructions for a computer to understand and execute. We need programming languages because computers only understand machine code (0s and 1s), and programming languages act as a bridge — translating human-readable instructions into commands the computer can carry out.

Q3. What is a Python interpreter?

Answer: A Python interpreter is a program that reads your Python code line by line and translates it into actions the computer can perform. When you click "Run" in your IDE, the interpreter reads each line of your code in order and executes it immediately.

Q4. What is a development environment? Name two IDEs used for Python.

Answer: A development environment is the collection of tools and software set up on a computer that allow a programmer to write, run, and debug code. Two popular IDEs (Integrated Development Environments) used for Python are **PyCharm** and **VS Code** (Visual Studio Code).

Q5. What is Google Colab? Why is it useful for beginners?

Answer: Google Colab is a free, cloud-based platform that provides a Jupyter notebook environment for writing and running Python code directly in a web browser. It is useful for beginners because it requires no installation — Python and popular libraries are already set up, so students can start coding immediately with just a Google account.

Q6. What important step must you follow when installing Python on Windows?

Answer: During installation, you must check the box that says **"Add Python to PATH."** This step tells the Windows operating system where to find Python, which makes it possible to run Python from the command line (terminal). If this step is skipped, Python may not work correctly from outside its installation folder.

Q7. What does it mean that Python is "versatile"?

Answer: Versatile means Python can be used for many different types of tasks. It is not limited to one field — programmers use Python for web development, data analysis, artificial intelligence, task automation, game development, and scientific research, among many other areas.

Section 2: Syntax, Structure, and Comments

Q8. What is syntax in Python?

Answer: Syntax refers to the set of rules that define how Python code must be written so that the Python interpreter can understand it. Just as grammar rules govern how sentences are structured in English, Python's syntax rules govern how instructions must be structured in code. Breaking syntax rules produces an error.

Q9. What does the `print()` function do? Give an example.

Answer: The `print()` function displays one or more values on the screen (in the output area). For example:

```
python
```

```
print("Hello, students!")
```

This will display: `Hello, students!`

Q10. What is a comment in Python? Why are comments used?

Answer: A comment is a line (or part of a line) in Python code that the interpreter completely ignores and does not execute. Comments are used to leave explanations, notes, or reminders inside the code for human readers — making programs easier to understand and maintain.

Q11. How do you write a single-line comment in Python? Give an example.

Answer: A single-line comment begins with the `#` (hash) symbol. Everything written after `#` on that line is ignored by Python. For example:

```
```python
```

## This program greets the user

```
print("Welcome!") # This prints the greeting
```
```

Q12. What is indentation in Python and why is it important?

Answer: Indentation refers to the spaces placed at the beginning of a line. In Python, indentation is not just for readability — it actually defines the structure and scope of the code (for example, which lines belong inside an `if` statement or a loop). Incorrect indentation causes Python to produce an `IndentationError`.

Q13. What is the difference between a syntax error and a logical error?

Answer: A syntax error occurs when Python cannot understand the code because a rule of the language has been broken (for example, a missing parenthesis). A logical error occurs when the code runs without crashing but produces the wrong result — because the programmer's logic was incorrect (for example, using `+` instead of `-` in a calculation).

Section 3: Variables and Data Types

Q14. What is a variable in Python? Give an example.

Answer: A variable is a named container in the computer's memory that stores a value. The value can be changed at any time during the program's execution. For example:

```
python
age = 16
print(age)    # Output: 16
age = 17
print(age)    # Output: 17
```

Q15. Why is it called a "variable" and not a "constant"?

Answer: It is called a variable because its value can vary (change) throughout the program. A constant, by contrast, holds a fixed value that never changes. Since Python allows you to reassign a new value to any variable at any point, the name "variable" reflects this flexibility.

Q16. What is the assignment operator? How is it different from the equality operator?

Answer: The assignment operator `=` stores a value inside a variable — for example, `x = 10` stores the number 10 in `x`. The equality operator `==` compares two values and returns `True` or `False` — for example, `x == 10` asks "is x equal to 10?" They look similar but perform completely different actions.

Q17. List three rules for naming variables in Python.

Answer:

1. A variable name must start with a letter (a–z, A–Z) or an underscore `_` — not a digit.
2. Variable names can only contain letters, digits, and underscores — no spaces or special characters like `-` or `@`.
3. Variable names are case-sensitive — `age`, `Age`, and `AGE` are three different variables.

Q18. Why can't Python's reserved keywords be used as variable names?

Answer: Reserved keywords are words that Python has already claimed for its own use — they form the core vocabulary of the language. Words like `if`, `while`, `for`, and `True` already mean something specific to Python. If you tried to use them as variable names, Python would become confused about whether you meant the keyword or the variable.

Q19. What are the four basic data types in Python? Give one example of each.

Answer:

- **int:** Whole numbers — `age = 16`
- **float:** Decimal numbers — `height = 1.75`
- **str (string):** Text — `name = "Fatima"`
- **bool (boolean):** True or False — `is_student = True`

Q20. What is the difference between `int` and `float` data types?

Answer: An `int` stores whole numbers with no decimal point, for example `age = 17`. A `float` stores numbers that include a decimal point, for example `price = 19.99`. In Python, even if a `float` has no fractional part (like `5.0`), it is still treated differently from the integer `5` — for example, division using `/` always returns a float.

Q21. What is a string in Python?

Answer: A string is a sequence of characters (letters, digits, spaces, symbols) enclosed in quotation marks. It represents text data. In Python, strings can be written with either single quotes `'hello'` or double quotes `"hello"`. For example: `name = "Ali"` stores the text Ali.

Q22. What is a boolean data type? What are its only two possible values?

Answer: A boolean is a data type that can hold only one of two values: `True` or `False`. It represents a logical condition — either something is true or it is not. Booleans are the result of comparison and logical operations. For example: `is_student = True`.

Q23. What does the `input()` function do?

Answer: The `input()` function displays a message (called a prompt) on the screen, then pauses the program and waits for the user to type something and press Enter. Whatever the user types is captured and returned as a string. For example:

```
name = input("Enter your name: ")
```

 stores whatever the user types into the variable `name`.

Q24. Why do we use `int()` or `float()` when taking numeric input?

Answer: Because `input()` always returns a string, even when the user types a number. If you try to do arithmetic with a string, Python gives an error. Wrapping the input in `int()` converts the string to an integer, and `float()` converts it to a decimal number, making mathematical operations possible.

Q25. Write a program that asks for a user's name and prints a greeting.

Answer:

```
python
name = input("Enter your name: ")
print("Hello, " + name + "! Welcome.")
```

Q26. Write a program that takes two numbers from the user and prints their sum.

Answer:

```
python
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
print("Sum:", a + b)
```

Section 4: Arithmetic Operators

Q27. What are arithmetic operators? List all seven with their symbols.

Answer: Arithmetic operators perform mathematical calculations. The seven arithmetic operators in Python are:

- `+` Addition
 - `-` Subtraction
 - `*` Multiplication
 - `/` Division (returns float)
 - `//` Floor Division (discards decimal)
 - `%` Modulus (returns remainder)
 - `**` Exponentiation (power)
-

Q28. What is the difference between `/` and `//` in Python?

Answer: The `/` operator performs true division and always returns a float, even if the result is a whole number — for example, `10 / 2` returns `5.0`. The `//` operator performs floor division and discards the decimal part, returning only the whole number — for example, `10 // 3` returns `3`, not `3.33`.

Q29. What is the modulus operator? Give a real-world use case.

Answer: The modulus operator `%` returns the remainder after dividing two numbers. For example, `11 % 4 = 3` because 11 divided by 4 gives 2 with a remainder of 3. A real-world use case is checking whether a number is even or odd: if `number % 2 == 0`, the number is even; otherwise it is odd.

Q30. What does the `**` operator do? Give an example.

Answer: The `**` operator raises the left number to the power of the right number (exponentiation). For example, `2 ** 8` means 2 raised to the power of 8, which equals 256. It is equivalent to multiplying 2 by itself 8 times.

Q31. What will the following code output? Explain why.

```
print(7 / 2)
print(7 // 2)
print(7 % 2)
```

Answer:

- `7 / 2` → `3.5` (true division, returns float)
- `7 // 2` → `3` (floor division, drops the decimal)
- `7 % 2` → `1` (remainder: $7 = 2 \times 3 + 1$)

Section 5: Comparison Operators

Q32. What are comparison operators? What do they always return?

Answer: Comparison operators compare two values and determine the relationship between them — such as whether they are equal, or which one is larger. They always return a Boolean value: either `True` or `False`. The six comparison operators are: `==`, `!=`, `>`, `<`, `>=`, and `<=`.

Q33. What is the purpose of the `!=` operator? Give an example.

Answer: The `!=` operator checks whether two values are NOT equal. If they are different, it returns `True`; if they are the same, it returns `False`. For example: `print(10 != 5)` outputs `True` because 10 and 5 are different.

Q34. What is the difference between `>` and `>=` operators?

Answer: The `>` (greater than) operator returns `True` only if the left value is strictly larger than the right — for example, `10 > 10` is `False`. The `>=` (greater than or equal to) operator returns `True` if the left value is larger OR equal — for example, `10 >= 10` is `True`.

Q35. Write a Python expression that checks whether a student's score of 45 is a passing grade (pass mark is 40).

Answer:

```
```python
score = 45
is_passing = score >= 40
print("Did the student pass?", is_passing)
```

**Output: Did the student pass? True**

```

Q36. Why is it a common mistake to write `=` instead of `==` in a comparison?

Answer: Because both operators use the equals sign, beginners often confuse them. `=` is the assignment operator — it stores a value (e.g., `x = 5` puts 5 into x). `==` is the comparison operator — it asks a question (e.g., `x == 5` asks "is x equal to 5?"). Using `=` where `==` is needed will cause an error or unexpected behaviour.

Section 6: Assignment Operators

Q37. What are compound assignment operators? Give two examples.

Answer: Compound assignment operators combine a mathematical operation with the assignment operator in one step. Instead of writing `a = a + 5`, you can write `a += 5`.

Two examples:

- `a += 10` means `a = a + 10`
- `a *= 3` means `a = a * 3`

Q38. If `score = 50`, what is the value of `score` after `score += 15`?

Answer: The value of `score` becomes `65`. The `+=` operator adds 15 to the current value of `score` (which is 50) and stores the result (65) back into `score`.

Q39. What is the output of the following code?

```
a = 10
a *= 2
a -= 3
print(a)
```

Answer: The output is `17`.

- `a = 10`
 - `a *= 2` → `a = 10 * 2 = 20`
 - `a -= 3` → `a = 20 - 3 = 17`
-

Q40. What does `a **= 3` mean?

Answer: It means raise `a` to the power of 3 and store the result back in `a`. It is equivalent to writing `a = a ** 3`. For example, if `a = 2`, then after `a **= 3`, the value of `a` becomes `8` (because $2^3 = 8$).

Section 7: Logical Operators

Q41. What are logical operators in Python? Name all three.

Answer: Logical operators are used to combine two or more Boolean (True/False) conditions into one final True/False result. The three logical operators in Python are:

- `and` — True only if BOTH conditions are True
 - `or` — True if at least ONE condition is True
 - `not` — Reverses (flips) a Boolean value
-

Q42. Explain the `and` operator with an example.

Answer: The `and` operator returns `True` only when both conditions on either side of it are True. If even one side is False, the result is False. For example:

```
python
score = 85
attendance = 80
eligible = (score >= 80) and (attendance > 75)
print(eligible)    # True – both conditions are True
```

Q43. Explain the `or` operator with an example.

Answer: The `or` operator returns `True` when at least one of the two conditions is True. It only returns `False` when both conditions are False. For example:

```
python
is_friday = False
is_prefect = True
gets_reward = is_friday or is_prefect
print(gets_reward)    # True – because is_prefect is True
```

Q44. Explain the `not` operator with an example.

Answer: The `not` operator flips a Boolean value — it turns `True` into `False` and `False` into `True`. For example:

```
python
x = True
print(not x)    # False
y = False
print(not y)    # True
```

Q45. What is the output of the following code?

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
```

Answer:

- `a and b` → `False` (both must be True; b is False)
 - `a or b` → `True` (at least one is True; a is True)
 - `not a` → `False` (flips True to False)
-

Q46. When does `False or False` evaluate to True?

Answer: It never does. `False or False` always evaluates to `False`. The `or` operator returns `True` only when at least one operand is `True`. Since both operands here are `False`, the result is `False`.

Q47. Create a real-life scenario where the `and` operator would be useful in a Python program.

Answer: A login system could use `and` to verify both a username and password: a user is granted access only if the username is correct AND the password is correct. If either one is wrong, access is denied. In Python:

```
access = (username == "admin") and (password == "12345").
```

Section 8: Operator Precedence

Q48. What is operator precedence in Python?

Answer: Operator precedence is the set of rules that determines which operator Python evaluates first when an expression contains multiple operators. Just like in mathematics where multiplication is done before addition, Python follows a strict order: Parentheses first, then Exponentiation, then Multiplication/Division/Modulus, then Addition/Subtraction.

Q49. What is the correct order of operator precedence in Python (from highest to lowest)?

Answer: The order from highest to lowest precedence is:

1. `()` — Parentheses
 2. `**` — Exponentiation
 3. `*`, `/`, `//`, `%` — Multiplication, Division, Floor Division, Modulus
 4. `+`, `-` — Addition and Subtraction
-

Q50. Why does `2 + 3 * 4` give 14 and not 20?

Answer: Because multiplication has higher precedence than addition. Python evaluates `3 * 4 = 12` first, then adds 2 to get `14`. To get 20, you would need to use parentheses: `(2 + 3) * 4`, which forces the addition to happen first.

Q51. How do parentheses affect operator precedence?

Answer: Parentheses have the highest precedence of all — whatever is inside parentheses is always evaluated first, regardless of the other operators present. This lets programmers override the default order and control exactly which calculation happens first. For example, `(2 + 3) * 4 = 20` vs `2 + 3 * 4 = 14`.

Q52. Evaluate step by step: `4 + 10 / 2 - 11 % 3`

Answer:

- Step 1: Division → `10 / 2 = 5.0`
 - Step 2: Modulus → `11 % 3 = 2`
 - Step 3: Addition → `4 + 5.0 = 9.0`
 - Step 4: Subtraction → `9.0 - 2 = 7.0`
 - **Final answer: 7.0**
-

Q53. What is an f-string in Python?

Answer: An f-string (formatted string) is a string prefixed with the letter `f` that allows you to embed variables or expressions directly inside the string using curly braces `{}`. Python automatically evaluates whatever is inside the braces and inserts the result. For example: `f"The answer is {2 + 3}"` produces the string `"The answer is 5"`.

Q54. What is meant by saying that `**` is "right-associative"?

Answer: Right-associative means that when multiple `**` operators appear in a row, Python evaluates them from right to left. For example, `2 ** 3 ** 2` is calculated as `2 ** (3 ** 2) = 2 ** 9 = 512` — not as `(2 ** 3) ** 2 = 64`. This matches standard mathematical convention for exponentiation.

Q55. What is the output of `print(3 + 2 ** 2 * 5)`? Show your working.

Answer:

- Step 1: Exponentiation → `2 ** 2 = 4`
 - Step 2: Multiplication → `4 * 5 = 20`
 - Step 3: Addition → `3 + 20 = 23`
 - **Output: 23**
-

Section 9: Mixed / Applied Questions

Q56. Write a Python program that takes a student's name and marks from the user, then prints whether they passed or failed (pass mark = 40).

Answer:

```
python
name = input("Enter student name: ")
marks = int(input("Enter marks: "))
passed = marks >= 40
print(name, "passed:", passed)
```

Q57. Explain the difference between `print("5 + 3")` and `print(5 + 3)`.

Answer: `print("5 + 3")` treats `5 + 3` as a string (text) and prints it exactly as written: `5 + 3`. The quotation marks tell Python this is just characters, not a calculation. `print(5 + 3)` treats `5 + 3` as an arithmetic expression, evaluates it, and prints the result: `8`.

Q58. What is the output of the following code? Explain each line.

```
x = 5
y = 3
print(x > y and y > 0)
print(x < y or y > 0)
print(not (x == y))
```

Answer:

- `x > y and y > 0` → `True and True` → **True** (both conditions hold)
 - `x < y or y > 0` → `False or True` → **True** (one condition holds)
 - `not (x == y)` → `not False` → **True** (5 is not equal to 3)
-

Q59. Why should a programmer use comments in their code?

Answer: Comments help anyone reading the code — including the programmer themselves in the future — understand what the code is doing and why. Well-commented code is easier to debug, maintain, and share with teammates. Comments also help break down complex logic into human-readable explanations without affecting the program's execution.

Q60. A student writes this code but gets an error. What is wrong and how should it be fixed?

```
age = input("Enter age: ")
new_age = age + 1
print(new_age)
```

Answer: The error is a `TypeError`. The `input()` function returns a string, and you cannot add the integer `1` to a string. The fix is to convert the input to an integer using

`int()`:

python

```
age = int(input("Enter age: "))
```

```
new_age = age + 1
```

```
print(new_age)
```

QUICK REFERENCE ANSWER KEY

MCQ Answer Key

| Q | Ans | Q | Ans | Q | Ans | Q | Ans |
|----|-----|----|-----|----|-----|----|-----|
| 1 | c | 21 | c | 41 | c | 61 | c |
| 2 | c | 22 | b | 42 | a | 62 | a |
| 3 | a | 23 | c | 43 | c | 63 | a |
| 4 | b | 24 | a | 44 | a | 64 | b |
| 5 | d | 25 | b | 45 | b | 65 | a |
| 6 | b | 26 | c | 46 | c | 66 | a |
| 7 | c | 27 | d | 47 | c | 67 | b |
| 8 | c | 28 | d | 48 | a | 68 | a |
| 9 | c | 29 | a | 49 | c | 69 | a |
| 10 | d | 30 | b | 50 | b | 70 | a |
| 11 | b | 31 | c | 51 | a | 71 | b |
| 12 | c | 32 | d | 52 | a | 72 | a |
| 13 | a | 33 | b | 53 | b | 73 | a |
| 14 | b | 34 | b | 54 | a | 74 | c |
| 15 | b | 35 | c | 55 | c | 75 | a |
| 16 | d | 36 | c | 56 | a | 76 | a |
| 17 | c | 37 | c | 57 | c | 77 | c |
| 18 | b | 38 | b | 58 | a | 78 | c |
| 19 | c | 39 | a | 59 | a | 79 | b |
| 20 | c | 40 | c | 60 | a | 80 | b |

End of Question Bank — Chapter 3: Introduction to Python Programming